Wrist IMU–Based Tremor Classification for Parkinson's Disease Screening

PBL on ECE - Final Presentation

Ahmed Sameh - 120230134
Faris Essam - 120230124
Mohab Harfoush - 120230112

Presented to:
Dr. Adel Bedair
Dr. Asano Tanemasa

Egypt - Japan University of Science and Technology

April 29, 2026

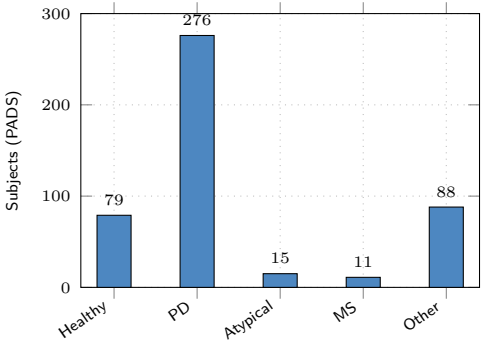
Outline

1. Problem Definition & Motivation
2. Solution & System Overview
3. Task-Specific Model Training
4. Hardware Implementation
5. Model Pipeline
6. Model Evaluation
7. Conclusion

The Clinical Problem

Key Facts

- Parkinson's Disease (PD) affects **10 million** people worldwide
- Average diagnostic delay: **4–6 years** after symptom onset
- Early detection improves treatment outcomes
- Gold-standard diagnosis requires specialist, DaTscan, and longitudinal observation



PADS Dataset - 469 subjects, 5 classes

Motivation

A low-cost wearable screening device could flag at-risk patients *before* clinical deterioration, routing them to specialists earlier.

Why Is This Hard?

The Core Challenge

All tremor conditions oscillate in the **same 4–6 Hz band**. Frequency alone does not separate them - and treating all motor tasks as interchangeable makes the problem worse.

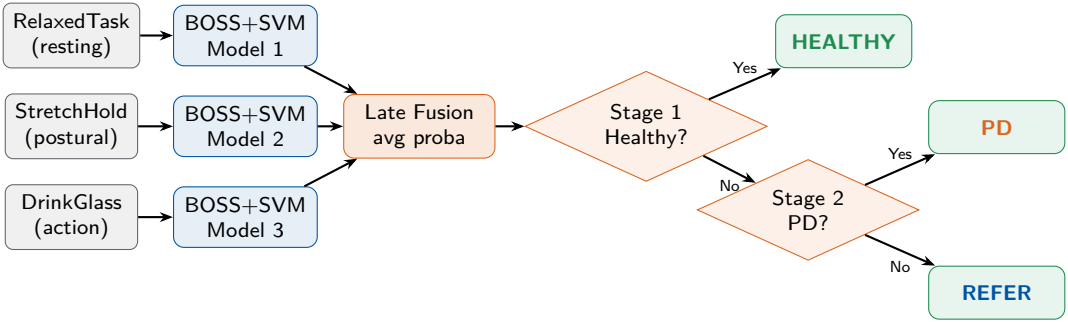
Why task segregation is required:

- Resting vs. postural difference requires *distinct clinical tasks*
- Task-specific signal structure: resting tremor $\neq$ postural tremor $\neq$ action tremor
- Standard single-stream models struggle to separate these varying physiological signals
- Bilateral asymmetry $\rightarrow$ needs 2 wrists

Modeling Complexity

Tremor pathologies are highly varied. Traditional global metrics obscure the exact localized oscillatory patterns needed to securely distinguish Parkinson's from Essential Tremor or other conditions.

Our Approach: Task-Guided Ensemble



Task-Specific Architecture

We utilize **three task-specific models**, each trained and tuned independently for its respective motor task.

Late Fusion

`predict_with_probs` outputs are averaged across tasks before the cascade - no feature-level merging.

Task-Specific BOSS Models

Why task-specific?

- Motor tasks produce qualitatively different signals
- Resting, postural, and action tremors have distinct spectral and temporal structures
- Specialized models prevent contradictory patterns from degrading learning

Window size rationale:

- `RelaxedTask`: smaller w for high-frequency resting tremor
- `DrinkGlass`: larger w for slow purposive movement
- `StretchHold`: intermediate w

Training procedure:

1. Collect raw data instances strictly for the targeted clinical task
2. Define task-specific parameter bounds for (l, α, w)
3. Train MultiBOSS ensemble over the delimited hyperparameter grid
4. Fit StandardScaler + downstream SVM classifier
5. Persist the isolated, task-optimized model pipeline

Late Fusion: Combining Task Predictions

Why late fusion over feature fusion?

Averaging `predict_with_probs` outputs keeps each task model fully independent. Feature concatenation would couple the hyperparameter spaces and require joint retraining whenever one task model changes.

Fusion procedure per subject:

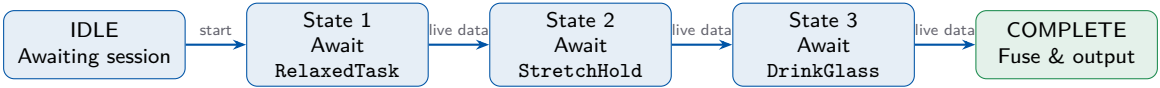
1. Retrieve `.npy` files for each of the 3 tasks
2. Run each file through its task-specific BOSS+SVM pipeline $\rightarrow$ probability vector $\mathbf{p}_k$
3. Average: $\bar{\mathbf{p}} = \frac{1}{K}\sum_{k=1}^{K} \mathbf{p}_k$
4. Feed $\bar{\mathbf{p}}$ into Stage 1 cascade decision
5. **Fallback:** if a subject is missing a task, average over remaining tasks only

Evaluation:

- `StratifiedKFold` grouped by `subject_id`
- Prevents any subject's windows appearing in both train and test
- Final metrics: macro F1, balanced accuracy, per-class precision / recall
- Robust assessment across 5 diagnostic classes

Live Inference State-Machine Protocol

From stateless inference to a structured clinical session:



Real-Time Controller:

- Ingests live data stream from the ESP32 hardware
- Loads all task-specific models into RAM at startup
- Maintains continuous per-session state
- Emits final ensemble diagnosis immediately after State 3

Hardware Integration:

- ESP32 buffer directly triggers model inference pipeline
- Completely seamless transition from physical movement to prediction vector
- Robust error handling for truncated or interrupted signal windows

Hardware Setup

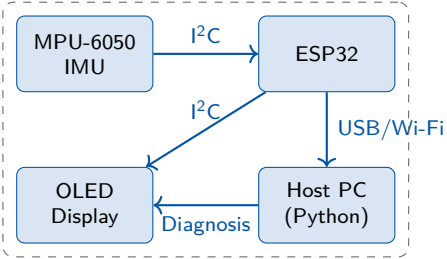
Components:

- ESP32 microcontroller
- MPU-6050 IMU (accel + gyro)
- Sampling rate: **100 Hz**
- Window: **1024 samples** (10 seconds)
- USB serial / BLE to host

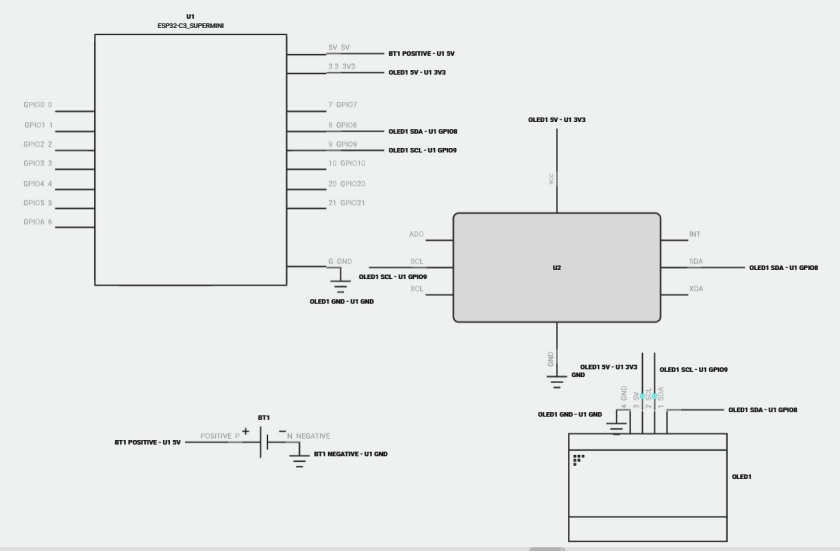
Data flow (per task):

1. IMU $\xrightarrow{I^2C}$ ESP32 buffer
2. Buffer fills (1024×6 samples)
3. Send over serial as .npy file
4. Host ingests live stream and decodes
5. State machine invokes rapid task-specific model

System Schematic



Hardware Schematic



Data Shapes & Pipeline

Per-Task Frame

Each task produces one 10-second window:

$$\underbrace{1024}_{\text{time steps}} \times \underbrace{6}_{\text{channels}} = 6{,}144 \text{ float32 values} \xrightarrow{\text{save}} \texttt{\{task\}.npy}$$

Channels: `aX, aY, aZ, gX, gY, gZ`

Per-task processing:

1. Reshape to $(6, 1024)$
2. DC removal + bandpass (0.5–20 Hz)
3. Task-specific BOSS $\rightarrow$ feature vector
4. Task-specific SVM $\rightarrow$ `predict_with_probs`
5. State machine stores $\mathbf{p}_k$

Shape summary:

Stage	Shape
Raw window	$(6, 1024)$
After BOSS	$(1, d),\ d \leq 42$
Per-task prob	$(1, C)$
Fused prob	$(1, C)$

C = number of classes; fusion averages $K \leq 3$ probability vectors

Why Dictionary-Based Classification?

Motivation

Tremor is not defined by a single frequency peak but by **repeating symbolic patterns**. Dictionary methods capture this naturally.

FFT / band-power (traditional approach):

- Summarises global frequency content
- Loses temporal ordering
- All tremor types share 4–6 Hz band
- Poor inter-class separability (confirmed by STFT)

Dictionary / symbolic approach:

- Captures *which patterns occur* and *how often*
- Encodes local shape, not just frequency
- Robust to amplitude variation across patients
- Works on raw or lightly pre-processed signal

Key Insight

PD tremor is **rhythmically regular**: the same short subsequence repeats. BOSS counts exactly these repetitions.

SFA: Symbolic Fourier Approximation

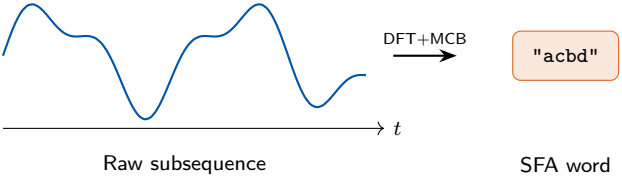
Step 1 - Dimensionality Reduction via DFT:

$$\hat{x}_1, \hat{x}_2, \ldots, \hat{x}_l = \mathrm{DFT}(x)[1{:}l], \quad l \ll T$$

Keep only the first l Fourier coefficients of each subsequence.

Step 2 - Quantisation (MCB):

- Fit breakpoints on training data distribution (per coefficient)
- Map each coefficient to one of α symbols $\in \{a, b, c, \ldots\}$
- Each subsequence becomes a **word** of length l



BOSS: Bag-of-SFA-Symbols

Step 3 - Build histogram over a full window:

1. Slide a window of length w (step 1) across the time series
2. Convert each position to an SFA word
3. **Numerosity reduction**: keep only the first occurrence of consecutive identical words
4. Count occurrences $\rightarrow$ **word histogram** $=$ feature vector

`acbd acbd bcda acbd cdab` ... (raw word stream)

↓

After numerosity reduction: `acbd bcda acbd cdab`

Histogram: {acbd: 2, bcda: 1, cdab: 1, ...} $\rightarrow$ **feature vector**

Complexity

Training: $\mathcal{O}(n \cdot T)$ Prediction: $\mathcal{O}(T)$ - fast enough for embedded deployment.

MultiBOSS: Ensemble over Hyperparameters

Problem: BOSS has three hyperparameters - word length l, alphabet size α, subsequence length w. No single setting is optimal for all patterns.

Solution - MultiBOSS:

1. Train BOSS classifiers for a *grid* of (l, α, w) combinations
2. Extract words across multiple timescales without discarding parallel models
3. **Concatenate** all extracted histograms into one master feature vector:

$$\mathbf{f} = \left[\mathrm{hist}_{(l_1,\alpha_1,w_1)} \parallel \mathrm{hist}_{(l_2,\alpha_2,w_2)} \parallel \cdots \right]$$

4. Feed the concatenated vector to a single downstream classifier

Why this matters for tremor

Tremor patterns exist at multiple timescales simultaneously. A short window captures oscillation shape; a long window captures rhythmic regularity. MultiBOSS learns both simultaneously.

SVM Classifier with StandardScaler Pipeline

sklearn Pipeline

```
Pipeline([("scaler", StandardScaler()),
    ("svm", SVC(kernel="rbf", class_weight="balanced"))])
```

Why StandardScaler?

- BOSS histograms have variable scales (common words: count $\gg 100$; rare: count $= 0$)
- RBF kernel uses Euclidean distance - unscaled features dominate
- $z = (x - \mu)/\sigma$, fitted on train only

Why SVM + RBF?

- BOSS histograms are sparse, high-dimensional
- RBF handles non-linear boundaries implicitly
- `class_weight="balanced"` corrects 21:1 class imbalance without oversampling

No Data Leakage

Scaler is inside the Pipeline - `fit` is called only on training data; val/test use `transform` only.

Evaluation Strategy

Group-level subject split:

- 469 subjects, `StratifiedKFold` grouped by `subject_id`
- All windows from one subject stay in the same fold
- Prevents patient leakage across splits
- Subjects missing a task: averaged over remaining tasks (not dropped)

Metrics:

- Macro F1 (primary)
- Balanced accuracy
- Per-class precision, recall, F1
- Cascaded end-to-end recall

Why not raw accuracy?

- Class ratio: 3312 PD vs. 180 Other
- Predicting PD always $\Rightarrow$ 73% accuracy
- Macro F1 penalises ignoring minority classes

Ensemble evaluation:

- Patient-level majority vote is applied within each task window set
- Late fusion happens *after* patient-level vote per task
- Follows the logical clinical workflow

Results

Stage 1: Healthy vs. Tremor

Class	Prec.	Recall
Healthy	0.40	0.42
Tremor	0.88	0.87
Macro F1	**0.643**	
Bal. Acc.	**0.647**	

End-to-End Cascaded Recall

Class	Detection Rate
Healthy	0.42
Parkinson's	$0.87 \times 0.79 = \mathbf{0.69}$
Other Tremor	$0.87 \times 0.32 = 0.28$
Macro Recall	**0.46**

Stage 2: PD vs. Other Tremor

Class	Prec.	Recall
Other Tremor	0.39	0.32
Parkinson's	0.74	0.79
Macro F1	**0.559**	
Bal. Acc.	**0.558**	

Validation

The ensemble model flags 87% of tremor patients for screening and isolates 69% of Parkinsonian tremors.

Limitations & Future Expansion

Current Scope Constraints:

- Single wrist classification limits structural tracking of bilateral asymmetry
- "Other Tremor" is a heterogeneous class combining 3 underlying conditions

Clinical Scaling:

- Validate cohort far beyond initial 469 parameters for definitive medical regulations

Next Technical Steps:

- Two-wrist network enabling synced bilateral cross-referencing
- Expand from 3 tasks to a comprehensive motor exam array
- Integrate Questionnaire data (NMS score) via stacking
- Full on-device standalone TFLite inference directly on ESP32-S3
- Longitudinal data extraction for continuous tracking

Screening vs. Diagnosis

This device is a **screening tool**. It simply answers: *"Should this person see a neurologist?"*

Conclusion

1. Built **three task-specific BOSS+SVM models** (`RelaxedTask`, `StretchHold`, `DrinkGlass`), each with independently tuned MultiBOSS window sizes
2. Implemented **late fusion** averaging `predict_with_probs` outputs across tasks before the two-stage cascade decision
3. Replaced stateless single-shot inference with a **sequential real-time state-machine protocol**, directly enabling seamless hardware integration and continuous evaluation.
4. Established evaluation under grouped `StratifiedKFold` with explicit missing-task fallback strategy

Thank you. Questions?

References

- Klucken et al. (2023). *PADS: Parkinson's Disease Smartwatch Dataset*. PhysioNet. doi:10.13026/xxx
- Schäfer, P. (2015). *The BOSS is concerned with time series classification in the presence of noise*. Data Mining and Knowledge Discovery, 29(6), 1505–1530.
- Löning et al. (2019). *sktime: A unified interface for machine learning with time series*. NeurIPS Workshop.
- Rahimian et al. (2022). *XceptionTime: Independent Time-Step Factorization for Deep Time Series Classification*. arXiv:2011.11956.